Compilers

Muchang Bahng

Spring 2026

Contents

1 Handwritten Scanners **4**
Example 1.1 (C Identifiers) . . . 4
Example 1.2 (Implementations in Languages) . . . 4
1.1 Token Types . . . 5
Example 1.3 (Parentheses) . . . 5
Example 1.4 (Braces) . . . 5
Example 1.5 (Brackets) . . . 5
Example 1.6 (Comma) . . . 5
Example 1.7 (Dot) . . . 5
Example 1.8 (Plus) . . . 6
Example 1.9 (Minus) . . . 6
Example 1.10 (Star) . . . 6
Example 1.11 (Star Star) . . . 6
Example 1.12 (Slash) . . . 6
Example 1.13 (In-Place Arithmetic) . . . 6
Example 1.14 (Backslash) . . . 7
Example 1.15 (Hashtag) . . . 7
Example 1.16 (At) . . . 7
Example 1.17 (Semicolon) . . . 7
Example 1.18 (Logic Operators) . . . 7
Example 1.19 (Equal) . . . 7
Example 1.20 (Greater, Less) . . . 7
Example 1.21 (Equal Equal, Not Equal, Less Equal, Greater Equal) . . . 8
Example 1.22 (Whitespace) . . . 8
Example 1.23 (Identifiers) . . . 8
Example 1.24 (String) . . . 8
Example 1.25 (Numbers) . . . 8
Example 1.26 (Boolean) . . . 8
Example 1.27 (Logic Keywords) . . . 8
Example 1.28 (Null Value) . . . 9
Example 1.29 (Control Flow) . . . 9
Example 1.30 (Functions) . . . 9
Example 1.31 (Varibale Declaration) . . . 9
Example 1.32 (Classes) . . . 9
Example 1.33 (Print) . . . 9
Example 1.34 (Begin, End) . . . 9
Example 1.35 (Comment) . . . 10
Example 1.36 (End of File) . . . 10
1.2 Scanner . . . 11
Example 1.37 (Ambiguities) . . . 11
Definition 1.1 (Maximal Munch) . . . 11
1.3 Scanner Struct . . . 12

2 Generated Scanners **17**
Example 2.1 (Ambiguities in Tokenizing) . . . 17

3 Parsing **18**
Example 3.1 (Precedence in the Parse Tree of $2 + 3 \times 5$) . . . 18
3.1 Precedence and Associativity . . . 18

Definition 3.1 (Precedence) . . . 18
Example 3.2 (Assignment) . . . 18
3.2 Abstract Syntax Trees . . . 18
Definition 3.2 (Expression) . . . 18
Definition 3.3 (Statement) . . . 18
Definition 3.4 (Declaration) . . . 19
Definition 3.5 (Categories of Words) . . . 19
Example 3.3 (Grammar Rule for jLox) . . . 19
3.3 Recursive Descent Parsing . . . 20
3.4 Bytecode and Pratt Parsing . . . 20
3.5 LL Parsing . . . 20
Definition 3.6 (LL Parsing) . . . 20
Definition 3.7 (Symbol Derives Empty) . . . 21
Example 3.4 (Nullable Non-Terminals) . . . 21
Definition 3.8 (Rule Derives Empty) . . . 21
Theorem 3.1 (Recursive Definition of SDE, RDE) . . . 21
Definition 3.9 (First Set) . . . 22
Example 3.5 (Computing FIRST Sets) . . . 22
Theorem 3.2 (Recursive Definition of FirstSet) . . . 22
Example 3.6 (Reading Off the Legal First Symbols) . . . 22
Definition 3.10 (Follow Set) . . . 23
3.6 LR Parsing . . . 23

4 Resolution **24**
Definition 4.1 (Scope) . . . 24

5 Type Checking **25**

6 Tree Walk Interpreters **26**
6.1 Evaluation of Expressions . . . 26
6.2 Environments . . . 26

7 Bytecode Interpreters **27**
Definition 7.1 (Bytecode) . . . 27
Example 7.1 (Lox Implementation of Bytecode Chunks) . . . 27
Definition 7.2 (Virtual Machine) . . . 27
7.1 Stack-Based Virtual Machines . . . 28
Definition 7.3 (Stack-Based Virtual Machine) . . . 28
Definition 7.4 (`OP_CONSTANT`) . . . 28
Definition 7.5 (Arithmetic) . . . 28
Definition 7.6 (Negation) . . . 28
Example 7.2 (Adding 2 Numbers) . . . 28
Definition 7.7 (`OP_RETURN`) . . . 29
7.2 Register-Based Virtual Machines . . . 29
Definition 7.8 (Register-Based Virtual Machine) . . . 29
Example 7.3 (Adding 2 Numbers) . . . 29

Compiling is the process of converting code that you write (essentially a giant string) into assembly, for a specific ISA. You can use a *cross-compilation toolchain*, where you have a machine for one ISA (e.g. x86) but compile it into ARM with—say the `gccxarm` package.

Languages themselves are not compiled or interpreted. Their implementations are. Second, the dichotomy between compiling and interpreting is much more grey.

1. *Lexer/Scanner.* Convert a sequence of characters into a sequence of tokens. Whitespace and comments are not tokens, since they get dropped out.
2. *Parsing.* Building the abstract syntax tree. e.g. LL parsing (what we are doing) vs LR parsing. MLyac. Trees make everything explicit and so is easier to work with.
3. *Type Checking.* Just a bit of work.
4. *IR.* Top of the mountain.
5. *Instruction Selection.*
6. *Liveness Analysis.* Data flow analysis, which is at the core of a lot of optimization.
7. *Register Allocation.* This gives the MIPS assembly, which is text.

1 Handwritten Scanners

If we look at our source code—whether it'd be in Python, Java, or C—they are all just a giant string. Every program is just a string composed of characters in an alphabet.[1] This motivates the following definitions.

Example 1.1 (C Identifiers)

The set $\Sigma = \{_, a, \dots, z, A, \dots, Z, 0, \dots, 9\}$ can be the alphabet of the formal language L representing all variable identifiers in the C programming language.

Example 1.2 (Implementations in Languages)

While the concept of "importing" other files into the current file is similar, the process is very different across languages.[a]

1. In C, preprocessing directives are a part of lexing. For example, `#include` literally copies and pastes the contents of the target module into the current file. With `#IFDEF`, `#IFNDEF`, `#ENDIF`, we can delete tokens from our file to do *conditional compilation*.
2. In Python, the `import` keyword is a runtime behavior done during execution of the statement. That is, doing something like

   ```
   import math
   math.sqrt(9)
   ```

   can be thought of as doing something like

   ```
   math = load_module("math")
   math.sqrt(9)
   ```

   It is basically like executing a function or instantiating a class.
3. In Java, the `import` keyword is closer to a name shortcut, done during resolution. These import statements allow us to write

   ```
   import java.util.ArrayList;

   public static void main(String arg) {
      ArrayList<Integer> x = new ArrayList<>();
   }
   ```

   rather than having to do

   ```
   public static void main(String arg) {
      java.util.ArrayList<Integer> x = new java.util.ArrayList<>();
   }
   ```

   Basically, it just tells Java that when I write `ArrayList`, I mean `java.util.ArrayList`. Now, it is clear that this is more of a name resolution process, where we are binding the `ArrayList` name to `java.util.ArrayList`.

[a] https://chatgpt.com/share/6a155f53-3404-83a7-8349-03818c8b23de

[1] Technically, not just English letters, but other ASCII characters plus other UTF-8.

1.1 Token Types

The immediate problem with this representation of our code is that our alphabet is too *fine*.[2] We need a slightly more restrictive alphabet, which give us a better building blocks of our language. In fact, choosing the set of token types is one of the first things you might want to think about when making a new language. Since we want to convert a series of characters (our program) into tokens, what tokens should we have in our language? Think about all the characters you have seen so far in programs, plus how they are used within and across different languages.

We start off with the single character tokens.

Example 1.3 (Parentheses)

Parentheses are used universally in almost all languages. We will label them `<LEFT_PAREN>` and `<RIGHT_PAREN>`.

1. *Precedence.* In C, Python, and Java, parentheses indicate some precedence in the order of operations, e.g. `(3 + 4) * 5`.
2. *Function Calls.* They also represent function calls, e.g. `int x = myFunc()`.
3. *Function Declaration.* They are used in function declarations, e.g. `public void myFunc(int x, int y) {...}`.

Example 1.4 (Braces)

Braces are also pretty universal. We will denote them `<LEFT_BRACE>` and `<RIGHT_BRACE>`.

1. *Block Delimiters.* In C, C++, and Java, braces represent the start and end of a block, whether it'd be in a function declaration, an if/else statement, or a loop.
2. *F-Strings.* In Python, braces can be used in f-strings, e.g. `name = "Bob"; print(f"Hello {name}")`.

Example 1.5 (Brackets)

For brackets, we will denote them `<LEFT_BRACKET>` and `<RIGHT_BRACKET>`.

1. *Indexing.* They are used to retrieve a value stored in the ith position of an array (in C, Java) or a list (Python).
2. *List Creation.* In Python, they are used to initialize a list, e.g. `x = [1, 2, 3]`.

Example 1.6 (Comma)

The comma token is denoted `<COMMA>`.

1. *Function Parameters.* You use commas to separate parameters in function declarations, e.g. `def myPythonFunction(x, y, z): ...`.
2. *Function Arguments.* You use commas to separate arguments in function calls, e.g. `int y = myFunc(4, 3);`.
3. *Unpacking.* When returning an iterable in Python, you can unpack them with commas, e.g. `x, *, y = someList`.

[2] In English, sentences are indeed made up of letters, but it would be better to represent them as a list of words.

Example 1.7 (Dot)

The dot token is denoted `<DOT>`.

1. *Attribute Retrieval.* In OOP, you can use it to retrieve an attribute from an object, e.g. `student.name`.
2. *Method Call.* In OOP, you can use it to call an instance or static method, e.g. `MyClass.Stuff()`.

Example 1.8 (Plus)

The plus token is denoted `<PLUS>`.

1. *Addition.* In almost all languages, you use this to add two numbers (int, double) together, e.g. `int x = y + z;`.
2. *Concatenation.* In Python, you use this to concatenate two strings into a longer string.

Example 1.9 (Minus)

The minus token is denoted `<MINUS>`.

1. *Subtraction.*

Example 1.10 (Star)

The star is denoted `<STAR>`.

1. *Multiplication.*
2. *Repeat.* In Python, you can multiply a string by an int to concatenate the string with itself multiple times, e.g. `"hello" * 4`.

Example 1.11 (Star Star)

The double star token is denoted `<STAR_STAR>`.

1. *Exponent.* In Python, it indicates an exponent, e.g. `3 ** 2`.

Example 1.12 (Slash)

The slash is denoted `<SLASH>`.

1. *Division.*

Example 1.13 (In-Place Arithmetic)

The following tokens are used commonly for in-place arithmetic.

1. *Increment.* We use the `<PLUS_PLUS>` token to represent increments, e.g. `i++`.
2. *Decrement.* We use the `<MINUS_MINUS>` token to represent decrements, e.g. `i-`.
3. *In-Place Addition.* We use the `<PLUS_EQUAL>` token to represent in-place addition, e.g. `i += 2`.
4. *In-Place Subtraction.* We use the `<PLUS_EQUAL>` token to represent in-place subtraction, e.g. `i -= 2`.
5. *In-Place Multiplication.* We use the `<PLUS_EQUAL>` token to represent in-place multiplication, e.g. `i *= 2`.

6. *In-Place Division.* We use the `<PLUS_EQUAL>` token to represent in-place division, e.g. `i /= 2`.

Example 1.14 (Backslash)

The backslash is denoted `<BACKSLASH>`.
1. *Escape Sequence.*

Example 1.15 (Hashtag)

The `<HASHTAG>` token is used to represent `#`.
1. *Comment.* In Python, this is used to represent single line comments, e.g. `# comment`.
2. *Directives.* In C and C++, it is used to represent preprocessing directive, e.g. `#include <stdio.h>` or `#IFDEF`.

Example 1.16 (At)

The `<AT>` token is used to represent .
1. *Macros.* In Python and Java, this is used to represent macros for methods and functions, e.g. `@property` or `@Override`.

Example 1.17 (Semicolon)

The semicolon is denoted `<SEMICOLON>`. As far as I know, it is used to terminate a line.

Next, we continue onto tokens that are one-character or two characters long.

Example 1.18 (Logic Operators)

The logic operators may be represented as these character sequences (like in C++ or Java), or as keywords (like in Python).
1. *Negation.* The `!` character represents the `<BANG>` token. In C, C++, and Java, we can take a boolean value and negate it, e.g. `!true`.
2. *Or.* The `||` characters often represent the `<OR>` token.
3. *And.* The `&&` characters often represent the `<AND>` token.

Example 1.19 (Equal)

The `<EQUAL>` token almost universally represents assignment, e.g. `x = 4`.

Example 1.20 (Greater, Less)

The `<LESS>` and `<GREATER>` tokens can represent the following.
1. *Comparison.* In almost all languages, these are used as comparison operators, e.g. `4 < 5`.
2. *Template Parameters.* In C++, you can use them as template parameters, e.g. `template <typename T>`.

3. *Element Type*. In Java, the types of the elements in the standard library data structures are specified within these angle brackets, e.g. `HashMap<String, Integer>`.

Example 1.21 (Equal Equal, Not Equal, Less Equal, Greater Equal)

The following four mean the same thing across all languages that I have seen so far.

1. *Equal Equal*. The `<EQUAL_EQUAL>` token is used to determine whether two values are equal.
2. *Not Equal*. The `<BANG_EQUAL>` token is used to determine whether two values are not equal.[a]
3. *Less Equal*. The `<LESS_EQUAL>` token is used to determine whether the previous token is less than the token after.
4. *Greater Equal*. The `<GREATER_EQUAL>` token is used to determine whether the previous token is greater than the token after.

[a]Note that this is completely separate from the concatenation of tokens `<NOT><EQUAL_EQUAL>`.

Example 1.22 (Whitespace)

In most languages, whitespace are not tokens. The exception is Python, which treat tabs as significant for identfying scope. We can label them as `<TAB>`.

Next, we talk about variable-length token types.

Example 1.23 (Identifiers)

Example 1.24 (String)

Example 1.25 (Numbers)

We can represent multiple types of numbers.

1. *Number*. The `<NUMBER>` token can be used to generically represent any number in your language. In Lox, this is the only token type.
2. *Integer*. The `<INTEGER>` token is used to represent integers.
3. *Float*. The `<FLOAT>` or `<DOUBLE>` token is used to represent floating point numbers.

Next, we talk about the keywords.

Example 1.26 (Boolean)

We can implement token types to represent boolean values `<TRUE>` and `<FALSE>`.

Example 1.27 (Logic Keywords)

Just as we have logic tokens, we have
1. *And*.
2. *Or*.
3. *Not*.

Example 1.28 (Null Value)

The null value may be represented depending on the language. It may be called `<NULL>`, `<NONE>`, or `<NIL>`.

Example 1.29 (Control Flow)

Control flow also has a bunch of keywords.
1. *Conditionals*. We have token types like `<IF>`, `<THEN>`, `<ENDIF>`, `<ELSEIF>`, and `<ELSE>`.
2. *For Loops*. We have token types like `<FOR>`, `<DO>`, `<ENDFOR>`.
3. *While Loops*. We have token types like `<WHILE>`, `<DO>`, `<ENDFOR>`.

Example 1.30 (Functions)

We have stuff like.
1. *Declaration*. Python uses `<DEF>` and JavaScript uses `<FUN>`.
2. *Return*. We have `<RETURN>`.

Example 1.31 (Varibale Declaration)

We can use `<VAR>` or `<LET>`.

Example 1.32 (Classes)

In object oriented languages, we can't forget about
1. `<CLASS>` used to declare a new class.
2. `<SUPER>` used to reference the parent class.
3. `<THIS>` or `<SELF>` used to reference the current instance of the class.

Example 1.33 (Print)

The `<PRINT>` token type may be used as a keyword. Often though, it is implemented as a native function.[a]

[a] More on this later.

Example 1.34 (Begin, End)

The <BEGIN> and <END> tokens are often used as delimiters of blocks rather than <LEFT_BRACE> and <RIGHT_BRACE>.

Example 1.35 (Comment)

Most of the times, comments are thrown away, but in certain cases one may want to keep them around as tokens. Then, we will have a <COMMENT>.

Example 1.36 (End of File)

Often, languages would implement an end-of-file token, denoted <EOF>.

The whole process of lexing is to convert a giant string (your code) into a sequence of tokens. But this requires us to classify which token the next substring is, i.e. whether it's an identifier, a keyword, a literal, etc. So basically, we need to match these incoming words to their respective token type, and we can do this by pattern matching. If you have thought of regular expressions to do this, you are exactly on the right track.

```
typedef enum {
   // Single-character tokens.
   TOKEN_LEFT_PAREN, TOKEN_RIGHT_PAREN,
   TOKEN_LEFT_BRACE, TOKEN_RIGHT_BRACE,
   TOKEN_COMMA, TOKEN_DOT, TOKEN_MINUS, TOKEN_PLUS,
   TOKEN_SEMICOLON, TOKEN_SLASH, TOKEN_STAR,
   // One or two character tokens.
   TOKEN_BANG, TOKEN_BANG_EQUAL,
   TOKEN_EQUAL, TOKEN_EQUAL_EQUAL,
   TOKEN_GREATER, TOKEN_GREATER_EQUAL,
   TOKEN_LESS, TOKEN_LESS_EQUAL,
   // Literals.
   TOKEN_IDENTIFIER, TOKEN_STRING, TOKEN_NUMBER,
   // Keywords.
   TOKEN_AND, TOKEN_CLASS, TOKEN_ELSE, TOKEN_FALSE,
   TOKEN_FOR, TOKEN_FUN, TOKEN_IF, TOKEN_NIL, TOKEN_OR,
   TOKEN_PRINT, TOKEN_RETURN, TOKEN_SUPER, TOKEN_THIS,
   TOKEN_TRUE, TOKEN_VAR, TOKEN_WHILE,

   TOKEN_ERROR, TOKEN_EOF
} TokenType;
```

Figure 1: Token types

Then, we can define a Token.

```
typedef struct {
   TokenType type;
   const char* start;
   int length;
   int line;
} Token;
```

Figure 2: Token struct.

1.2 Scanner

Now that we have regex's, we can just pattern match for each substring. However, we have some ambiguities.

Example 1.37 (Ambiguities)

If you are trying to lex the string:

```
!=
```

You can either tokenize it as `<BANG_EQUAL>` or `<BANG><EQUAL>`. This creates an ambiguity in our tokenizer.

Definition 1.1 (Maximal Munch)

The principle of tokenizing the maximum length substring at a given time is called **maximal munch**.

We employ this using a classic two pointer strategy.[3]

Algorithm 1.1 (Scanner)

The general idea is to use a two pointer approach, where `i` represents the start index of the current token and `j` represents the current index of the current token. Some tips for implementation:

```
procedure SCAN(source)
    tokens ← []
    i ← 0
    j ← 0
    while j < source.length() do
        c = jth character of source
        if c ∈ { ( , ) , {, } , , , . , - , + , ; , * } then          ▷1-char tokens
            convert c to corresponding token and append it to tokens
            increment j
        else if c == / then          ▷It could be a division, comment, or multiline comment
            c2 ← jth character of source    ▷We peek ahead to find out which token / is a part of
            if c2 = / then                                  ▷If this is single-line comment
                Keep incrementing j until we hit a newline.
                Increment j once more
            else if c2 = * then                             ▷If this is multiline comment
                Keep incrementing j until we hit */
                Increment j once more
```

[3] I initially tried it using a stack, but I found out later than a two pointer was much easier and faster.

```
            else                                                    ▷This must be a division
                Add <SLASH> token
                Increment j
            end if
        else if c = ! then
            Peek ahead and add either <BANG_EQUAL> or <BANG>
            Increment j past this token.
        else if c = = then
            Peek ahead and add either <EQUAL_EQUAL> or <EQUAL>
            Increment j past this token.
        else if c = > then
            Peek ahead and add either <GREATER_EQUAL> or <GREATER>
            Increment j past this token.
        else if c = < then
            Peek ahead and add either <LESS_EQUAL> or <LESS>
            Increment j past this token.
        else if c = " then
            Keep incrementing j until we hit "
            Increment j past this token.
        else if c is a digit then
            Keep incrementing j until we hit the end of the number.
            Increment j past this token.
        else if c is a letter then
            Keep incrementing j as long as the character is alphanumeric or underscore.
            If substring from i to j is a keyword, add the corresponding token.
            Otherwise, this is an identifier.
        end if
    end while
    Append <EOF> token to tokens.
    return tokens
end procedure
```

Note that the maximal munch principle is always applied in every conditional statement except for the first branch with 1-character tokens.

1. For the 1-character tokens, you can just use a switch statement which is faster, and to detect the identifiers/keywords/numbers, we can use an if statement in the default.
2. To see keyword vs identifier, you usually use a hashmap to check for keywords (e.g. `if` → `<IF>`), and if there are no matches, then you catch it with a regex.

1.3 Scanner Struct

The current index `j` keeps track of the column, but it might be nice to keep a second variable `line` tracking the current line. With this idea, let's start implementing our scanner.

```c
typedef struct {
    const char* start;
    const char* current;
    int line;
} Scanner;

void initScanner(const char* source) {
    scanner.start = source;
    scanner.current = source;
    scanner.line = 1;
}
```

We'll instantiate a `Scanner` object once and have these functions return a token with the current length.

```
static Token makeToken(TokenType type) {
   Token token;
   token.type = type;
   token.start = scanner.start;
   token.length = (int)(scanner.current - scanner.start);
   token.line = scanner.line;
   return token;
}

static Token errorToken(const char* message) {
   Token token;
   token.type = TOKEN_ERROR;
   token.start = message;
   token.length = (int)strlen(message);
   token.line = scanner.line;
   return token;
}
```

Then we'll have some helper functions for our scanner.

```
static bool isAtEnd() {
   return *scanner.current == '\0';
}

static char advance() {
   scanner.current++;
   return scanner.current[-1];
}

static char peek() {
   return *scanner.current;
}

static char peekNext() {
   if (isAtEnd()) return '\0';
   return scanner.current[1];
}

static bool match(char expected) {
   if (isAtEnd()) return false;
   if (*scanner.current != expected) return false;
   scanner.current++;
   return true;
}

static void skipWhitespace() {
   for (;;) {
      char c = peek();
      switch (c) {
         case ' ':
         case '\t':
         case '\r':
   scanner.current++;
   break;
         case '\n':
   scanner.current++;
```

```
    scanner.line++;
          case '/':
             if (peekNext() == '/') {
                // A comment goes until the end of the line.
                while (peek() != '\n' && !isAtEnd()) advance();
             } else {
                return;
             }
             break;
          default:
    return;
       }
    }
}
```

The next is for strings, which is pretty simple and standalone.

```
static Token string() {
   // we are currently at the start quote
   while (peek() != '"' && !isAtEnd()) {
      if (peek() == '\n') scanner.line++;
      advance();
   }
   if (isAtEnd()) return errorToken("Unterminated string.");
   advance();
   return makeToken(TOKEN_STRING);
}
```

For numbers, we use a helper method.

```
static bool isDigit(char c) {
   return c >= '0' && c <= '9';
}

static Token number() {
   while (isDigit(peek())) advance();

   // floating point number
   if (peek() == '.' && isDigit(peekNext())) {
      // consume the '.'
      advance();
      while (isDigit(peek())) advance();
   }

   return makeToken(TOKEN_NUMBER);
}
```

For identifiers, we use a lookahead to determine if it is a keyword. It is basically a double switch statement which makes things fast.

```
static bool isAlpha(char c) {
   return (c >= 'a' && c <= 'z') || (c >= 'A' && c <= 'Z') || c == '_';
}

static TokenType checkKeyword(int start, int length, const char* rest, TokenType type) {
   if (scanner.current - scanner.start == start + length &&
         memcmp(scanner.start + start, rest, length)) {
```

```
        return type;
    }
    return TOKEN_IDENTIFIER;
}

static TokenType identifierType() {
    switch (scanner.start[0]) {
        case 'a': return checkKeyword(1, 2, "nd", TOKEN_AND);
        case 'c': return checkKeyword(1, 4, "lass", TOKEN_CLASS);
        case 'e': return checkKeyword(1, 3, "lse", TOKEN_ELSE);
        case 'i': return checkKeyword(1, 1, "f", TOKEN_IF);
        case 'f':
            if (scanner.current - scanner.start > 1) {
                switch (scanner.start[1]) {
                    case 'a': return checkKeyword(2, 3, "lse", TOKEN_FALSE);
                    case 'o': return checkKeyword(2, 1, "r", TOKEN_FOR);
                    case 'u': return checkKeyword(2, 1, "n", TOKEN_FUN);
                }
            }
            break;
        case 'n': return checkKeyword(1, 2, "il", TOKEN_NIL);
        case 'o': return checkKeyword(1, 1, "r", TOKEN_OR);
        case 'p': return checkKeyword(1, 4, "rint", TOKEN_PRINT);
        case 'r': return checkKeyword(1, 5, "eturn", TOKEN_RETURN);
        case 's': return checkKeyword(1, 4, "uper", TOKEN_SUPER);
        case 't':
            if (scanner.current - scanner.start > 1) {
    switch(scanner.start[1]) {
        case 'h': return checkKeyword(1, 2, "is", TOKEN_THIS);
        case 'r': return checkKeyword(1, 2, "ue", TOKEN_TRUE);
    }
            }
            break;
        case 'v': return checkKeyword(1, 2, "ar", TOKEN_VAR);
        case 'w': return checkKeyword(1, 4, "hile", TOKEN_WHILE);
    }
    return TOKEN_IDENTIFIER;
}

static Token identifier() {
    while (isAlpha(peek()) || isDigit(peek())) advance();
    return makeToken(identifierType());
}
```

Finally, we bring it all together to our scan function.

```
Token scanToken() {
    skipWhitespace();
    scanner.start = scanner.current;
    if (isAtEnd()) return makeToken(TOKEN_EOF);

    char c = advance();

    if (isAlpha(c)) return identifier();
    if (isDigit(c)) return number();

    switch(c) {
        case '(': return makeToken(TOKEN_LEFT_PAREN);
```

```
13          case ')': return makeToken(TOKEN_RIGHT_PAREN);
14          case '{': return makeToken(TOKEN_LEFT_BRACE);
15          case '}': return makeToken(TOKEN_RIGHT_BRACE);
16          case ';': return makeToken(TOKEN_SEMICOLON);
17          case ',': return makeToken(TOKEN_COMMA);
18          case '.': return makeToken(TOKEN_DOT);
19          case '-': return makeToken(TOKEN_MINUS);
20          case '+': return makeToken(TOKEN_PLUS);
21          case '/': return makeToken(TOKEN_SLASH);
22          case '*': return makeToken(TOKEN_STAR);
23          case '!':
24             return makeToken(
25                   match('=') ? TOKEN_BANG_EQUAL : TOKEN_BANG);
26          case '=':
27             return makeToken(
28                   match('=') ? TOKEN_EQUAL_EQUAL : TOKEN_EQUAL);
29          case '<':
30             return makeToken(
31                   match('=') ? TOKEN_LESS_EQUAL : TOKEN_LESS);
32          case '>':
33             return makeToken(
34                   match('=') ? TOKEN_GREATER_EQUAL : TOKEN_GREATER);
35          case '"': return string();
36       }
37
38       return errorToken("Unexpected character.");
39    }
```

2 Generated Scanners

It turns out that using handwritten lexers is not the most powerful nor practical way to do this. Remember that all scanning does is pattern matching using regular expressions. In modern days, we have software that can take in whatever set of regular expressions you give it, and it will convert it to an equivalent DFA. Therefore, to detect tokens, we run it through a DFA and see the accepting state it lands on.

Again, with multiple matches, the problem is manifested with multiple dead states. If it is in a dead state, then output the most recent[4] accepting state.

Example 2.1 (Ambiguities in Tokenizing)

Say that we have a language with the following tokens.
1. `<x>: x.`
2. `<nxy1>: (x*)y`
3. `<z>: z`

This leads to the following DFA.

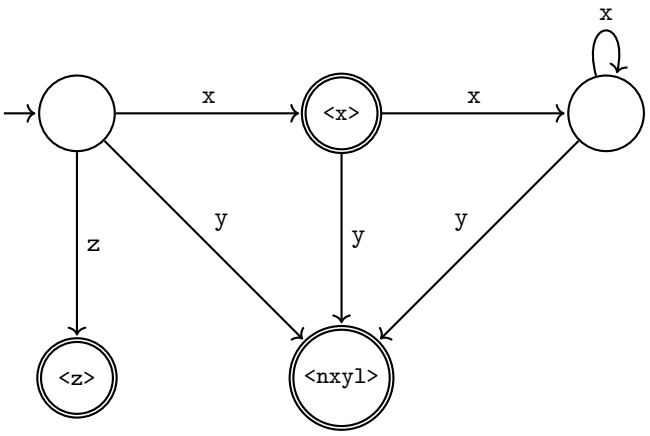


Figure 3

Then, to parse "`xxxzxxxy`," we have 3 choices, which leads to ambiguity.
1. `<x> <x> <x> <z> <nxy1>`
2. `<x> <x> <x> <z> <x> <nxy1>`
3. `<x> <x> <x> <z> <x> <x> <nxy1>`

Again, we can use maximal munch.

Lexers can have multiple DFAs, and depending on context (through start state), you traverse different DFAs (e.g. code vs comments).

[4] due to principle of maximal munch

3 Parsing

Once we are done scanning, we now have a list of tokens (t_n). The next job is to determine if this list of tokens is valid syntax for our language, i.e. $t_1 t_2 \dots t_n \in L$. So, the parser's first job is to determine membership of our sequence of tokens. This seems like a pretty hard problem since L can be very complex.

NOTE: Introduce parsing, then next 2 subsections should be the final result of what we want the parsed output to look like, i.e. either an AST or bytecode.

We basically want not only inclusion but a parse tree. But a sequence of tokens may have 2 valid parse trees.

Example 3.1 (Precedence in the Parse Tree of $2 + 3 \times 5$)

For example, `2 + 3 * 5` should really be translated to

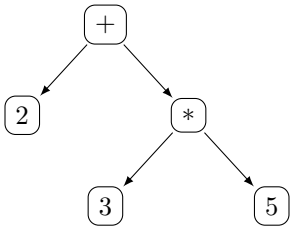


Figure 4: Syntax tree for the expression $2 + 3 * 5$.

We could sometimes refactor our grammar or sometimes use precedence directives.

3.1 Precedence and Associativity

Definition 3.1 (Precedence)

Precedence is an ordering on operators that determines which operator becomes deeper in the parse tree when an expression contains multiple possible parses. When an operator has higher precedence than another, it is said to **bind more tightly**.

Example 3.2 (Assignment)

Assignment is usually right associative.

$$\texttt{a = b = c = d} \iff \texttt{a = (b = (c = d))} \tag{1}$$

3.2 Abstract Syntax Trees

Parse trees are grammar level structure while AST is implementation level structure. The three major AST node types are:

Definition 3.2 (Expression)

An **expression** is a piece of syntax that can be *evaluated* to produce a value.

Definition 3.3 (Statement)

A **statement** is a piece of syntax that can be *executed* to produce an effect.

Definition 3.4 (Declaration)

A **declaration** is a piece of syntax that is used to introduce a name or binding.

Definition 3.5 (Categories of Words)

In a language,

1. a **literal** is a word in L that represents a fixed value directly in the source code.
2. an **identifier** is a word in L used to name entities such as variables, functions, and types.
3. a **reserved word**, or **keyword**, is an identifier that has a fixed meaning in the language's grammar.

Example 3.3 (Grammar Rule for jLox)

```
program     -> declaration* EOF
declaration -> varDecl
      | funDecl
      | statement;
varDecl     -> "var" IDENTIFIER = exprStmt
funDecl     -> "fun" function;
function    -> IDENTIFIER "(" parameters? ")" block ;
parameters  -> IDENTIFIER ("," IDENTIFIER)* ;
statement   -> exprStmt
      | printStmt
      | Block
      | ifStmt
      | whileStmt
      | forStmt
      | returnStmt
block       -> "{" declaration* "}";
exprStmt    -> expression ";"
printStmt   -> "print" expression ";"
ifStmt      -> "if" "(" expression ")" statement ("else" statement )? ;
whileStmt   -> "while" "(" expression ")" statement;
forStmt     -> "for" "(" (varDecl | exprStmt | ) ";" expression? ";" expression? ")"
    statement;
returnStmt  -> "return" expression? ";" ;
expression  -> assignment;
assignment  -> IDENTIFIER "=" assignment
       | logic_or;
logic_or    -> logic_and ( "or" logic_and )*
logic_and   -> equality ( "and" equality )*
equality    -> comparison ( ( "==" | "!=" ) comparison )*
comparison  -> term       ( ( ">" | ">=" | "<" | "<=" ) term )*
term        -> factor     ( ( "-" | "+" ) factor )*
factor      -> unary      ( ( "/" | "*" ) unary )*
unary       -> ( "!" | "-" ) unary
```

```
        | call
call        -> primary ( "(" arguments? ")")*;
arguments   -> expression ("," expression)*;
primary     -> NUMBER
    | STRING
    | "true"
    | "false"
    | "nil"
    | "(" expression ")"
    | IDENTIFIER
```

3.3 Recursive Descent Parsing

The simplest form of parsing, depending on how you do this, this will determine associativity for sure. To resolve ambiguities, recursive descent mostly does grammar refactoring.

Parsing expressions. Should be okay since we defined precedence and associativity before. Parsing statements. Parsing declarations.

3.4 Bytecode and Pratt Parsing

Another top-down (LL-like) parsing method that outputs an array of bytecode rather than an AST.

```
typedef struct {
  Token previous;
  Token current;
} Parser;
```

3.5 LL Parsing

The whole purpose of parsing is to derive an order of operations of our tokens. There are two types or parsing. We will start out with *LL parsing*, which is easier to construct by hand.

Definition 3.6 (LL Parsing)

LL parsing refers to parsing while reading the input from left-to-right, with leftmost derivation, aka "top-down" or "recursive descent." As the name suggests, we write a bunch of mutually recursive functions—one function per non-terminal symbol. Any time we need to parse the nonterminal symbol, we call that function.

Let us take the grammar $G = (V, \Sigma, R, S)$ with nonterminals $V = \{S, C, A, B, Q\}$, terminals $\Sigma = \{x, a, d, b, q\}$, start symbol S, and production rules. Note that for any symbol X, $X \to$ is shorthand for $X \to \epsilon$, i.e. the rule that can "delete" X without any cost. Furthermore, \$ indicates an *end of input*.

1. $\mathtt{S} \to AC\$$
2. $C \to c$
3. $C \to$
4. $A \to aBCd$
5. $A \to BQ$
6. $B \to bB$

7. $B \to$
8. $Q \to q$
9. $q \to$

Ideally, we would like an algorithm to parse these things similar to the functions below (`eat(c)` is a function that consumes the next token and requires it to match `c`).

```
parseS():
  parseA()
  parseC()
  eat("$")
parseC():
  if (peek() == "x"):
    eat("x")
```

```
parseA():
  if (peek() == "a"):
    parseB()
    parseC()
    eat("d")
  else:
    parseB()
    parseQ()
```

The potential problem is that there may be multiple choices for rules. To fix this, we talk about 4 things: *SDE*, *RDE*, *First Set*, *Follow Set*

Definition 3.7 (Symbol Derives Empty)

A **SDE** is a function that takes in a non-terminal and returns a boolean, indicating true if the non-terminal symbol could derive the empty string.

Example 3.4 (Nullable Non-Terminals)

In the example above, B, Q can both derive empty since $B \implies \epsilon, Q \implies \epsilon$. Consequently, since $A \implies BQ$, A can also derive empty. S cannot since it has a "$."

Definition 3.8 (Rule Derives Empty)

An **RDE** is a function that takes in a rule $(\alpha \to \beta)$ and returns a boolean, indicating true if the rule can be used to derive the empty string.

Theorem 3.1 (Recursive Definition of SDE, RDE)

Let $\wedge, \vee$ is the logical AND and OR operations, respectively. Then,

$$\text{SDE}(S) = \bigvee_{S \to *} \text{RDE}(S \to *) \tag{2}$$

For nonterminal n and terminal t,

$$\text{RDE}(N \to *t*) = 0 \tag{3}$$

since t must be in all future derivations. Given nonterminals $R_0, \dots, R_n$, we have

$$\text{RDE}(n \to R_0 \dots R_n) = \bigwedge_{i=0}^{n} \text{SDE}(R_i) \tag{4}$$

This is a good definition in itself mathematically, but this has no guarantee of termination. We can implement

this fix by using a graph algorithm, keeping track of the visited states, and saying that if a symbol ever derives back to itself, then we return False. A different way is to iterate to a *fixed point*, which is a great algorithmic approach in general.

Algorithm 3.1 (Iterative Computation of SDE, RDE)

The general idea is that you have a system of equations and want a solution to it. You start with a solution and keep refreshing it until it doesn't change. Start by assuming no symbols derive empty and no rules deriving empty. You iteratively correct this assumption by looking at the rules, starting with the simplest (e.g. $C \to \epsilon$ and $Q \to \epsilon$).

Basically, this is a 0th order optimization problem in $\{0,1\}^N$. This will terminate since once you switch over to True, you can't go back to false.

Definition 3.9 (First Set)

Given nonterminal n and terminal t, the `FirstSet` function takes in any string of nonterminals/terminals and returns a set of terminals. More specifically, a terminal t' is in

$$\texttt{FirstSet}((n|t)^*) \tag{5}$$

if and only if there exists some derivation that has its first character as t'.

Example 3.5 (Computing FIRST Sets)

In the example above, we can compute

$$\text{FirstSet}(aBqD) = \{a\} \tag{6}$$
$$\text{FirstSet}(B) = \{b, q\} \tag{7}$$
$$\text{FirstSet}(BqD) = \{b, q\} \tag{8}$$

Theorem 3.2 (Recursive Definition of FirstSet)

We can define

$$\text{FirstSet}() = \{\} \tag{9}$$
$$\text{FirstSet}(t(n|t)^*) = \{t\} \tag{10}$$
$$\text{FirstSet}(n(n|t)^*) = \begin{cases} \bigcup_{n\to *} \text{FirstSet}(*) & \text{if SDE}((n|t)^*) = 0 \\ \text{FirstSet}((n|t)^*) \cup \bigcup_{n\to *} \text{FirstSet}(*) & \text{if SDE}((n|t)^*) = 1 \end{cases} \tag{11}$$

Note that the $\bigcup_{n\to *} \text{FirstSet}(*)$ may infinitely recurse. So to actually compute this, we again do an iterative approach, designed so that it is guaranteed to converge.

Algorithm 3.2 (Iterative Computation of FirstSet)

Example 3.6 (Reading Off the Legal First Symbols)

So after computing this, we can see that a string in this language must start with an a, b, q, c, or \$, but not with a d!

The next function answers the following. Given that I have a nonterminal, what can legally come after it for any valid word in my language?

Definition 3.10 (Follow Set)

Let $G = (V, \Sigma, R, S)$ be a grammar. Then for any nonterminal $n \in V$, $\text{FollowSet}(n) \subset \Sigma$ is the set of terminals satisfying the following property: $t \in \text{FollowSet}(n)$ if there exists a some word w derived from S that contains a n immediately followed by a t. That is,

$$S \implies w, \quad w = \dots nt \dots \tag{12}$$

3.6 LR Parsing

Shift (=incrementing) reduce (package into Expr) conflict. For example, `if then ... if then ... else ...`. Is this shift or reduce?

Parser generators allow a compact ambiguous grammar, but they use precedence (and associative) directives, which is an easier way to annotate this rather than writing down 10 more layers of grammar rules. This resolves shift-reduce conflicts.

4 Resolution

The scope refers to where a name is visible.

Definition 4.1 (Scope)

The **scope** of a variable is the region of code where the name is visible.

The resolution is basically a syntactical analysis step where we make sure which value does a given variable refer to? We call this *resolving a variable*.

Later on, we will see this implemented through an **environment**, which is a runtime data structure that stores the name-to-value bindings.

5 Type Checking

This isn't needed for dynamically typed languages, but for statically typed ones, we need this. This is *semantic analysis*.

6 Tree Walk Interpreters

This is the simplest way to run source code, but it is so slow and memory heavy. Very few modern language implementations use source code interpreters. What is extremely common is that they are compiled to an intermediate form, and then the intermediate form is interpreted in some form of VM.

6.1 Evaluation of Expressions

6.2 Environments

7 Bytecode Interpreters

Java and Python run on a bytecode VM. The process is that after you scan, you take the list of tokens and convert them into bytecode.

Definition 7.1 (Bytecode)

Bytecode is a low-level compact instruction sequence—where each instruction/**opcode** is usually encoded in a byte—produced from source code and consumed by a virtual machine or interpreter.

The fact that each instruction is a byte limits us to a total of 256 instructions. Usually, this is enough but sometimes, we may need to apply variable-length encoding to instructions.

Example 7.1 (Lox Implementation of Bytecode Chunks)

In the Lox language, a *chunk* is an object that stores a piece of compiled bytecode. The two most important parts are that it contains

1. `ValueArray constants`, which is a dynamically resizable array of heap-allocated elements of type `Value` (which is basically just a double). It serves to contain elements that may be bigger than 1 byte and is most analogous to the read-only data that C uses to store literals.
2. `uint8_t* code`, which is an array of bytes. Each byte can represent either an opcode or the index in `constants` where the literals are stored.

```
typedef double Value;

typedef struct {
  int capacity;
  int count;
  Value* values;
} ValueArray;

typedef struct {
  int count;      // number of bytes in our array 'code'
  int capacity;   // number of bytes we have allocated in 'code'
  uint8_t* code;  // bytes of opcode or the index in 'constants'
  int* lines;     // array keeping track of each byte's line number
  ValueArray constants; // constant pool containing objects bigger than 1 byte
} Chunk;

```

Figure 5

Ultimately, the compiler must try to convert.

Definition 7.2 (Virtual Machine)

A **virtual machine** is a tuple $(T, \texttt{ip})$ that contains

1. a pointer to the sequence of tokens generated by the scanner.
2. an **instruction pointer** `ip` that points to the next opcode.[a]

[a] Note that this instruction pointer is specific to the VM, not the actual CPU.

There are two main types of virtual machines: stack-based and register-based. These are based off of the execution model, not whether the final machine code uses stacks or registers.

7.1 Stack-Based Virtual Machines

The key to stack-based virtual machines is the realization that traversing a tree (AST) is really just the same thing as DFS, which uses a stack. Therefore, all the evaluations and executions can be done with a stack.

Definition 7.3 (Stack-Based Virtual Machine)

A **stack-based virtual machine** is a virtual machine $(T, \texttt{ip}, S)$ that contains an extra stack S.

```c
#define STACK_MAX 256

typedef struct {
  Chunk* chunk;
  uint8_t* ip;
  Value stack[STACK_MAX];
  Value* stackTop;
} VM;

```

Figure 6: C implementation of a stack-based virtual machine.

This allows us to define the relevant opcodes.

Definition 7.4 (`OP_CONSTANT`)

The `OP_CONSTANT` is a bytecode instruction that tells the VM to load a constant value from the chunk's constant pool and push it on to the VM stack.

Definition 7.5 (Arithmetic)

The arithmetic operators all bytecode instructions that tell the VM to do the same thing.

1. `OP_ADD` says to pop 2 values, add them, and push the result.
2. `OP_SUBTRACT` says to pop 2 values, subtract them, and push the result.
3. `OP_MULTIPLY` says to pop 2 values, multiply them, and push the result.
4. `OP_DIVIDE` says to pop 2 values, divide them, and push the result.

Definition 7.6 (Negation)

The `OP_NEGATE` is a bytecode instruction that tells the VM to pop 1 value, negate it, and push the result to to stack.

It is often nice to see the state of the stack within each instruction. This is called the **stack trace**.

Example 7.2 (Adding 2 Numbers)

```
Instruction      Stack

OP_CONSTANT 0    [1]
OP_CONSTANT 1    [1, 2]
OP_ADD           [3]
```

Definition 7.7 (`OP_RETURN`)

The `OP_RETURN` is a bytecode instruction that tells the VM to return the current value(?)

7.2 Register-Based Virtual Machines

Definition 7.8 (Register-Based Virtual Machine)

A **register-based virtual machine** is a virtual machine $(T, \texttt{ip}, R)$ that contains an extra set of *registers*.

```c
#define REGISTER_MAX 256

typedef struct {
  Chunk* chunk;
  uint8_t* ip;
  Value registers[REGISTER_MAX];
} VM;

```

Figure 7: C implementation of a register-based virtual machine.

Example 7.3 (Adding 2 Numbers)

```
LOAD_CONST r0, 0
LOAD_CONST r1, 1
ADD r2, r0, r1
RETURN r2
```